Prompt engineering techniques

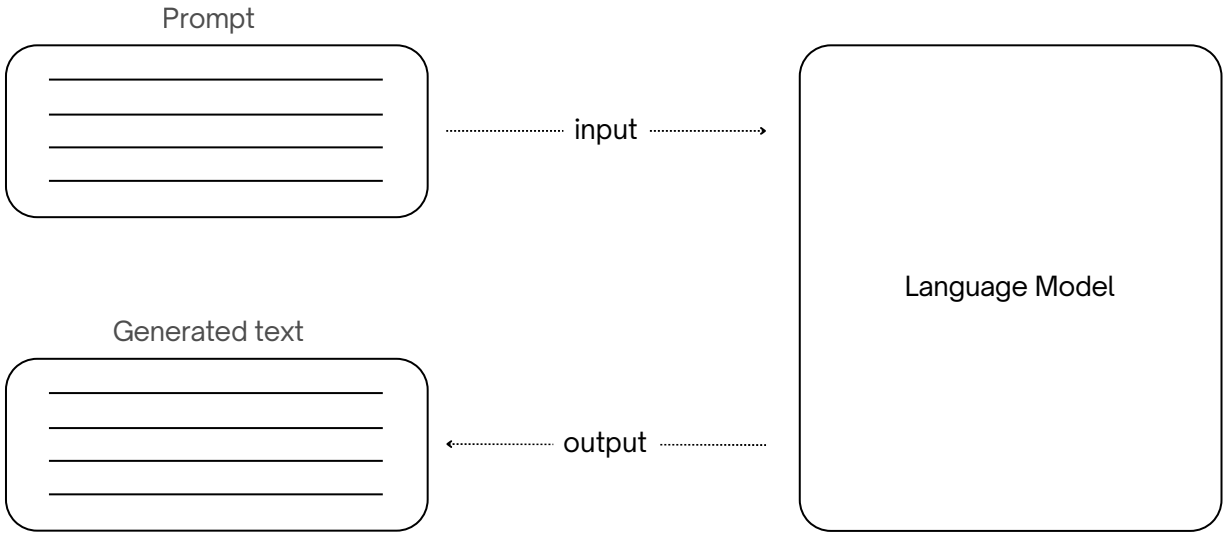
By Aleksander Obuchowski

What is prompt engineering?

Prompt engineering refers to the process of crafting effective and engaging prompts for artificial intelligence language models. It involves designing input instructions or queries that elicit desired outputs or responses from the model. Prompt engineering plays a crucial role in leveraging the capabilities of language models like ChatGPT to generate accurate and relevant information.

To better understand prompt engineering, let's take a look at an example. Suppose you have an AI language model and want it to generate a creative story based on a given theme. In this case, prompt engineering involves designing a prompt that clearly defines the theme, provides any necessary constraints (e.g., the story should be set in a specific time period), and even includes relevant examples or details to guide the model's imagination. It's like giving the model a vivid picture of the story you want it to create, so it can paint it with words.

Analogously, prompt engineering can be compared to building instructions for a LEGO masterpiece. Just as you need to provide step-by-step guidance to assemble a specific LEGO set, prompt engineers need to break down complex tasks into smaller, more manageable instructions for the AI model. It's like giving the model a set of LEGO bricks and telling it how to arrange them to build a specific object or scene.



Prompt engineering is important because it helps us get the best and most accurate results from AI language models. Here are a few reasons why prompt engineering matters:

- **Precision and Relevance**: Well-crafted prompts ensure that AI models understand what we want them to do or answer. By providing clear and specific instructions, we increase the chances of receiving accurate and relevant responses. It's like giving the model a map to follow so it doesn't get lost and provides us with the correct information.
- **Contextual Understanding:** AI models need context to generate appropriate responses. Prompt engineering allows us to provide that context by framing the question or task in a way that the model understands. It's like explaining a story from the beginning so that someone who doesn't know anything about it can still understand and respond accordingly.
- **Task Customization**: Different tasks require different prompts. By carefully engineering prompts, we can customize the behaviour of AI models to suit specific needs. Whether it's generating a story, answering a question, or translating text, prompt engineering allows us to guide the model towards the desired outcome. It's like giving someone different instructions depending on what we want them to do.
- **Quality Control**: Prompt engineering plays a role in ensuring the quality and reliability of AI-generated outputs. By refining and testing prompts, we can identify and address any biases, inaccuracies, or unintended behaviours exhibited by the model. It's like checking and correcting mistakes in a homework assignment to make sure it's accurate and well-written.

LLMs Parameters

LLMs are not just black boxes that magically produce content. They are intricate systems guided by a set of parameters that control their behavior and the quality of their outputs. Understanding these parameters is crucial for anyone looking to harness the full potential of LMMs

Model Size: The performance of a pre-trained language model depends on its size. Larger models tend to produce better quality outputs but are slower and more expensive. For example, the GPT4 model is much larger than GP3 but also more expensive. Smaller models are cheaper and faster, but they're not as powerful, so they're better used for simpler tasks, like classification, while the larger models are useful for creative content generation.

Number of Tokens: Language models generate a list of tokens (roughly 4 characters, but not always) and their probabilities as outputs. You can set a limit to how many tokens are generated to prevent the model from generating outputs indefinitely. Smaller models can go up to 1024 tokens while larger models go up to 32k tokens. However, it's generally recommended to generate in short bursts versus one long burst.

Temperature: This parameter determines how creative the model should be. A temperature of 0 makes the model deterministic, meaning it will always choose the word with the highest probability. As you increase the temperature, the model starts to consider words with lower probabilities. For instance, at a temperature of 5, it's biased towards lower probabilities and therefore more creative but might return statements that are not factually correct.

Frequency and Presence Penalties: These parameters help to control repetition in your outputs. The frequency penalty penalizes tokens that have already appeared in the preceding text, and scales based on how many times that token has appeared. The presence penalty applies the penalty regardless of frequency. As long as the token has appeared once before, it will get penalized.

General tips for prompt engineering

Break the task down: LLMs often perform better if the task is broken down into smaller steps, e.g. first ask the model to extract relevant information to process in

example

Your goal is to summarise the following article into a short description
1. Generate a table of contents (TOC)
2. For each item in TOC write a short summary of 2-3 sentences
3. Write a summary of the whole article

Add clear syntax: use some clear separator (like "###" or “---”) to separate different parts of the prompt

example

Instructions
Translate the text from English to French
Text
A quick brown fox jumps over the lazy dog

Avoid Impreciseness: Be very specific about the instruction and task you want the model to perform. The more descriptive and detailed the prompt is, the better the results

example

Bad Example: "Write something about dogs."
Good Example: "Write a 200-word informative paragraph about the dietary needs of adult Golden Retrievers."

Clarity over restriction: avoid saying what not to do but say what to do instead, e.g. instead of saying "Don't use any colloquial words" say "The text should be formal"

example

Bad Example: "Don't use any big words in your explanation of quantum physics."
Good Example: "Explain quantum physics in simple, everyday language suitable for a high school student."

Confirm whether conditions are met: The prompt can be designed to verify assumptions first. This is particularly helpful when dealing with edge cases. For example, if the input text doesn't contain any instructions, you can instruct the model to write "No steps provided".

example

Bad Example: "Summarize the steps in the text."

Good Example: "If the text contains any steps, summarize them. If not, write 'No steps provided'."

Prime the output: Include a few words or phrases at the end of the prompt to obtain a model response that follows the desired form.

example

###Intructions###
Write an essay describing why mitochondria is the powerhouse of the cell
###Essay###
Firstly let's consider the structure of the cell

Prompt Engineering Frameworks

A prompt framework is a structured approach or set of guidelines used to construct prompts for language models. It provides a systematic way to communicate with the model and specify the desired task, context, and format for generating responses. These frameworks aim to ensure clarity, consistency, and effective communication between humans and language models.

Prompt frameworks typically consist of different components, such as role, task, steps, context, examples, and format. Each component serves a specific purpose in conveying the intended instructions and requirements to the model. By following a prompt framework, users can provide clear and explicit guidance to the language model, increasing the chances of obtaining desired and relevant responses.

The framework helps users define a specific role for the model to assume, such as an expert, critic, or any other relevant persona. It states the task or objective to be accomplished, such as summarizing text, answering questions, or generating code. The framework may also include step-by-step instructions or an outline to guide the model's thought process and generate well-structured responses.

RTF

R - Role: In the prompt, we pretend to be someone or take on a specific role to give the model a better understanding of who it is interacting with. It's like playing a character in a story to make the conversation more engaging and meaningful.

T - Task: We state a specific task or goal that we want the model to accomplish. This can be things like summarizing a text or writing an article on a given topic. By clearly defining the task, we guide the model's focus and intention.

F - Format: We define the desired format or structure of the output. For example, if we want the model to generate code, we specify that. If we need the output in a specific format like CSV (a type of spreadsheet file), we mention that as well. Defining the format ensures that the model generates output in the desired way.

example

You are a travel blogger who recently visited an enchanting island called Serenity Cove. It's known for its pristine beaches, lush tropical forests, and vibrant marine life.

Your task is to write a captivating paragraph that highlights the unique features and attractions of Serenity Cove, enticing your readers to plan a visit. Remember to incorporate the island's breathtaking landscapes, adventurous water sports, and local cultural experiences.

Write the paragraph in plain text

example

You are a data analyst working for a research firm, tasked with extracting specific information from a set of financial reports.
Extract the company names, their respective revenue figures, and the corresponding fiscal years from a collection of annual financial reports.
Output the results in CSV (Comma-Separated Values) format, with columns for "Company Name," "Revenue," and "Fiscal Year."

CTF

C - Context: We provide context for the task, explaining any relevant background information or specific conditions. This helps the model understand the situation better and generate more appropriate responses. It's like giving the model additional information to work with.

T - Task: We state a specific task or goal that we want the model to accomplish. This can be things like summarizing a text or writing an article on a given topic. By clearly defining the task, we guide the model's focus and intention.

F - Format: We define the desired format or structure of the output. For example, if we want the model to generate code, we specify that. If we need the output in a specific format like CSV (a type of spreadsheet file), we mention that as well. Defining the format ensures that the model generates output in the desired way.

example

You receive an email from a frustrated customer who is experiencing issues with a product and is seeking assistance. Your goal is to provide a helpful and empathetic response to address the customer's concerns.

Draft a reply email to the customer, acknowledging their frustrations and offering a solution or troubleshooting steps to resolve their product issues. Ensure that your response conveys empathy, professionalism, and a commitment to resolving the problem.

Format the response as an email with

RTSCEF

R - Role: In the prompt, we pretend to be someone or take on a specific role to give the model a better understanding of who it is interacting with. It's like playing a character in a story to make the conversation more engaging and meaningful.

T - Task: We state a specific task or goal that we want the model to accomplish. This can be things like summarizing a text or writing an article on a given topic. By clearly defining the task, we guide the model's focus and intention.

S - Steps: We outline the steps or instructions for the task. It's like giving the model a set of clear directions or a recipe to follow. Breaking down the task into steps helps the model understand the process and perform the desired task more effectively.

C - Context: We provide context for the task, explaining any relevant background information or specific conditions. This helps the model understand the situation better and generate more appropriate responses. It's like giving the model additional information to work with.

E - Examples: We give the model examples of input and output to show what we expect. By providing examples, we illustrate the desired behaviour and help the model learn the patterns and structure of the desired output.

F - Format: We define the desired format or structure of the output. For example, if we want the model to generate code, we specify that. If we need the output in a specific format like CSV (a type of spreadsheet file), we mention that as well. Defining the format ensures that the model generates output in the desired way.

Advanced prompting techniques

Chain-of-thought

Chain-of-thought prompting is a method used to enhance the reasoning capabilities of language models. Here's a simplified explanation:

Imagine you're trying to solve a problem. You wouldn't just jump to the answer right away, right? Instead, you'd think through the problem step by step, considering different aspects and possibilities before arriving at your final answer. This process of thinking through a problem is what we call a "chain of thought".

Now, let's apply this concept to language models. Normally, when we ask a language model a question, it gives us an answer based on its training, but it doesn't show us the steps it took to arrive at that answer. Chain-of-thought prompting changes this. It's a way of asking the model to not only give an answer but also to provide a step-by-step explanation of how it arrived at that answer. This is like asking the model to show its "chain of thought".

This method has been found to be particularly effective with larger models (those with around 100 billion parameters). For smaller models, chain-of-thought prompting may result in fluent but illogical chains of thought, leading to lower performance than standard prompting.

The benefits of chain-of-thought prompting include:

- It can improve the model's performance on reasoning tasks, especially those that have flat scaling curves with standard prompting.
- It can help the model generalize to longer sequence lengths in symbolic reasoning tasks.
- It can be applied to a wide range of tasks, making it a versatile method for enhancing a model's reasoning abilities.
-

However, it's important to note that while chain-of-thought prompting can emulate the thought processes of human reasoners, it doesn't necessarily mean that the model is actually "reasoning" in the way humans do. Also, the method is more effective with larger models, which can be costly to use in real-world applications. Future research could explore how to induce reasoning in smaller models.

Zero-shot chain of thought (Zero-shot-CoT) prompting is the simplest version of CoT prompting. In the context of machine learning, "zero-shot" refers to the ability of a model to handle tasks that it hasn't explicitly seen during training. When we say "Zero-Shot Chain of Thought (Zero-shot-CoT)" prompting, it means that we're asking the model to generate a chain of thought or a step-by-step explanation for a question, even if it hasn't been specifically trained to do so.

The model is not given any specific examples or training on how to generate these chains of thought during its initial training phase. Instead, it leverages its general understanding of language and knowledge learned during its training to generate these explanations. This is why it's referred to as "zero-shot" - it's able to perform this new task without any specific examples or training.

It uses a remarkably straightforward zero-shot prompt. By simply adding "Let's think step by step." to a question, large language models (LLMs) can create a sequence of reasoning that provides an answer. This reasoning sequence allows for the extraction of more precise answers.

example

What is the process of photosynthesis? Let's think step by step

example

Can you explain the process of how a bill becomes a law in the United States, starting from the introduction of the bill in Congress, through the committee review and voting process, to the President's approval or veto, and finally how a veto can be overridden? Let's think step by step.

Few-shot Chain of Thought (Few-shot-CoT) prompting is similar to Zero-shot-CoT, but with a key difference: it uses a few examples (known as "shots") during the prompting process to guide the language model's responses.
In Few-shot-CoT, you provide the model with a few examples of the kind of output you want before you ask your main question. These examples serve as a guide for the model, showing it the kind of step-by-step reasoning you're looking for.
For instance, if you're asking the model to explain a complex process, you might first provide a few examples of similar processes explained step by step. This helps the model understand the format and level of detail you're looking for in its response.
The main advantage of Few-shot-CoT over Zero-shot-CoT is that it can potentially provide more accurate and detailed responses since the model has some examples to guide its reasoning. However, it also requires more effort to set up, since you need to provide relevant examples for each prompt.

example

Question: "Explain the process of photosynthesis in plants. Let's think step by step."
Response: "First, the plant takes in sunlight, which is absorbed by chlorophyll in the leaves. Second, the plant absorbs carbon dioxide from the air and water from the soil. Third, these elements undergo a chemical reaction in the chloroplasts, producing glucose, which is used for the plant's energy, and oxygen, which is released back into the air."

Question: "Describe how a refrigerator works. Let's think step by step."
Response: "First, the refrigerant absorbs heat from the inside of the fridge, cooling down the air. Second, the refrigerant evaporates and is drawn into the compressor. Third, the compressor pressurizes the refrigerant, causing it to heat up. Fourth, the heated refrigerant moves through the coils on the back of the fridge, releasing its heat into the room and cooling down again. Finally, the refrigerant returns to the inside of the fridge, and the cycle starts over."

Question: "Can you explain the process of how a bill becomes a law in the United States? Let's think step by step."
Response:

Automatic Chain of Thought (Auto-CoT) works by automatically generating examples, or "demonstrations," of how to solve a particular type of problem. These demonstrations consist of a question, a reasoning process, and an answer.

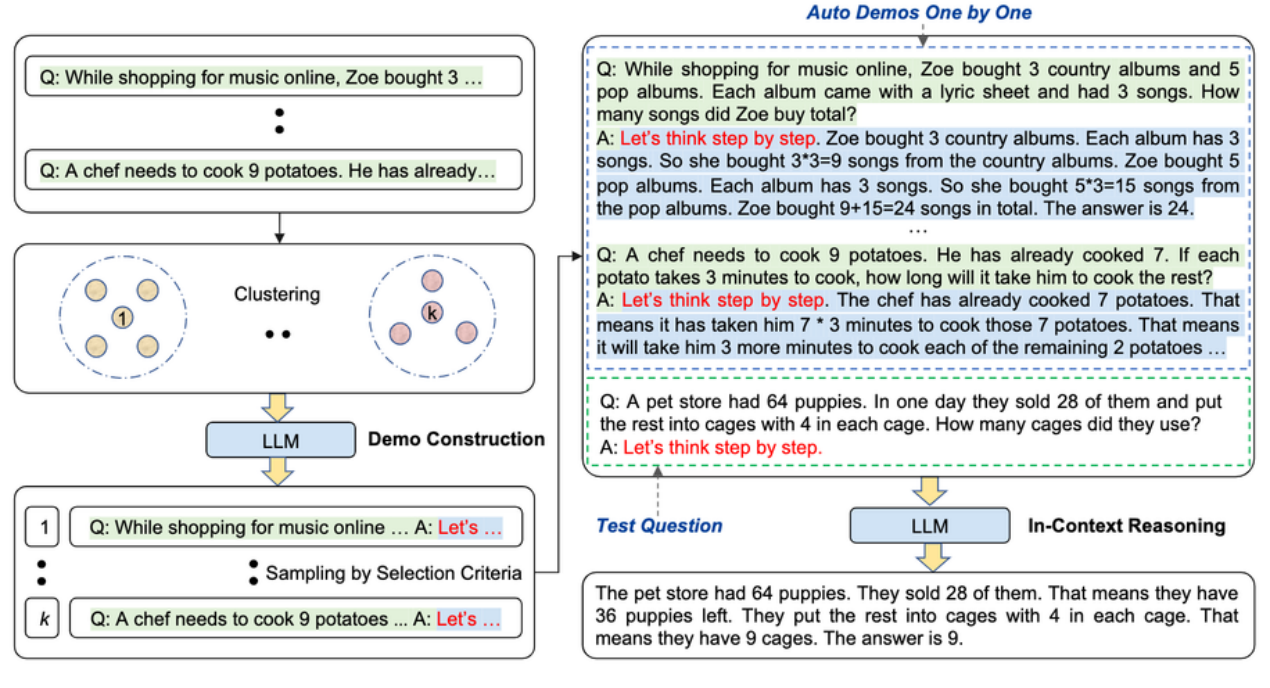
This method uses a clustering algorithm to group similar questions together and then selects a diverse set of questions from each cluster to use as demonstrations. This helps ensure that the language model learns to solve a wide range of similar problems, rather than just memorizing specific examples

Auto-CoT first clusters the given questions based on their similarity, and then selects a diverse set of questions from each cluster to form the demonstrations. The diversity is measured by the distance between the questions in the embedding space.

For each selected question, Auto-CoT generates a reasoning chain that consists of a sequence of intermediate steps leading to the final answer. The reasoning chain is generated by invoking a language model with the question and a prompt that guides the model to generate the intermediate steps.

Once the demonstrations are constructed, they are used to prompt the language model to perform in-context reasoning on new test questions. The language model is trained to generate the intermediate steps and the final answer given the question and the demonstration.

Experimental results on ten public benchmark reasoning datasets showed that with GPT-3, Auto-CoT consistently matches or exceeds the performance of the CoT paradigm that requires manual designs of demonstrations



read more

📝 Zhang, Zhuosheng, Aston Zhang, Mu Li, and Alex Smola. "Automatic chain of thought prompting in large language models." arXiv preprint arXiv:2210.03493 (2022).

Reasoning without observation (ReWOO)

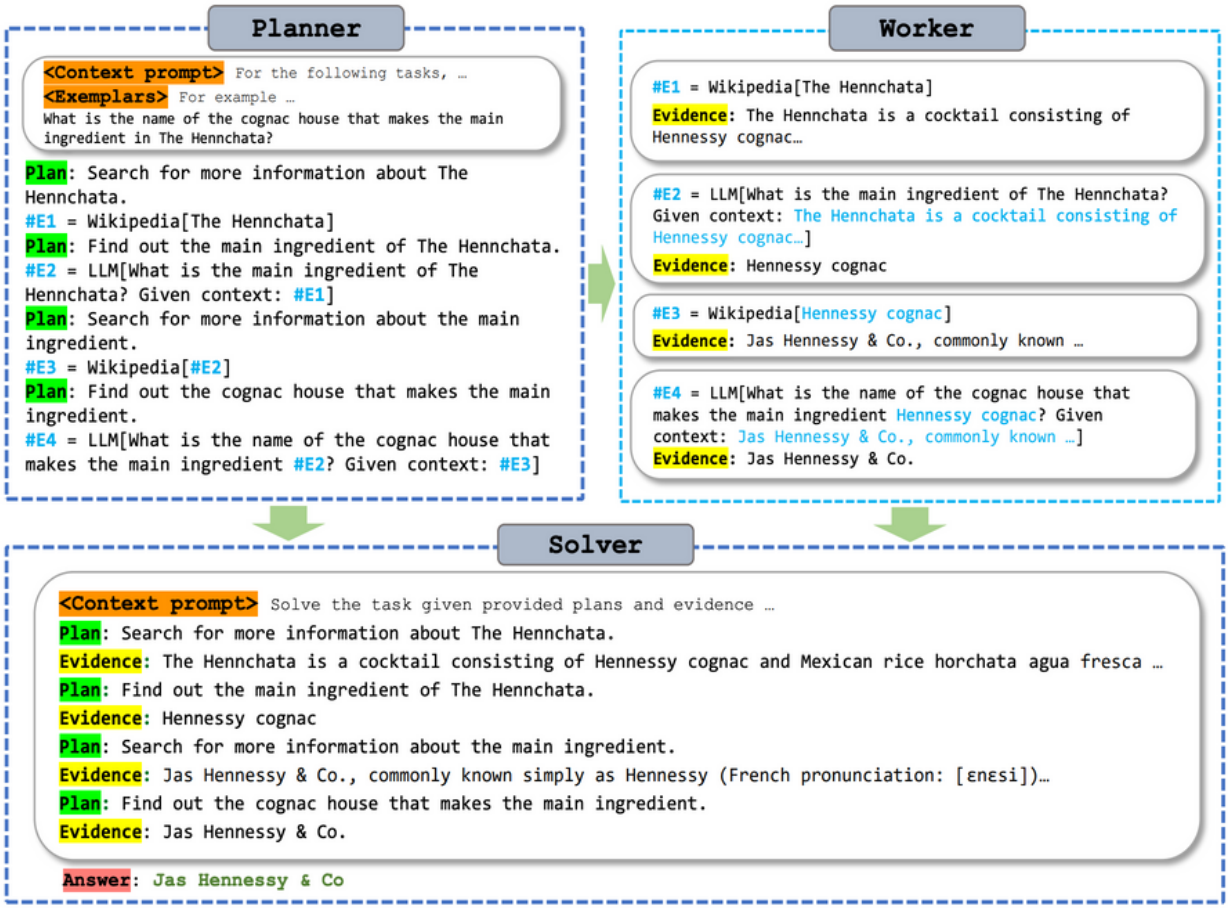
ReWOO is a framework that uses language models to solve complex tasks by breaking them down into smaller steps. The framework has three main components: Planner, Worker, and Solver.

Planner uses a language model to create a plan for solving the task, which includes a list of steps and the evidence (#Es) needed for each step.

Worker interacts with external tools to gather the evidence needed for each step. Once Planner provides a blueprint, designated Workers are invoked with instruction input, and populate #Es with real evidence or observations.

Solver then uses the evidence gathered by Worker to solve the task.

ReWOO is designed to be more efficient than other similar frameworks by reducing the number of prompts needed and improving performance.



read more

📝 Xu, Binfeng, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. "ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models." arXiv preprint arXiv:2305.18323 (2023).

🖥️ https://github.com/billxbf/ReWOO

ReAct

ReAct, short for Reasoning + Acting, is a method that draws inspiration from the way humans combine "acting" and "reasoning" to learn new tasks and make decisions. It's a way to enhance the capabilities of Large Language Models (LLMs) by integrating reasoning and action.

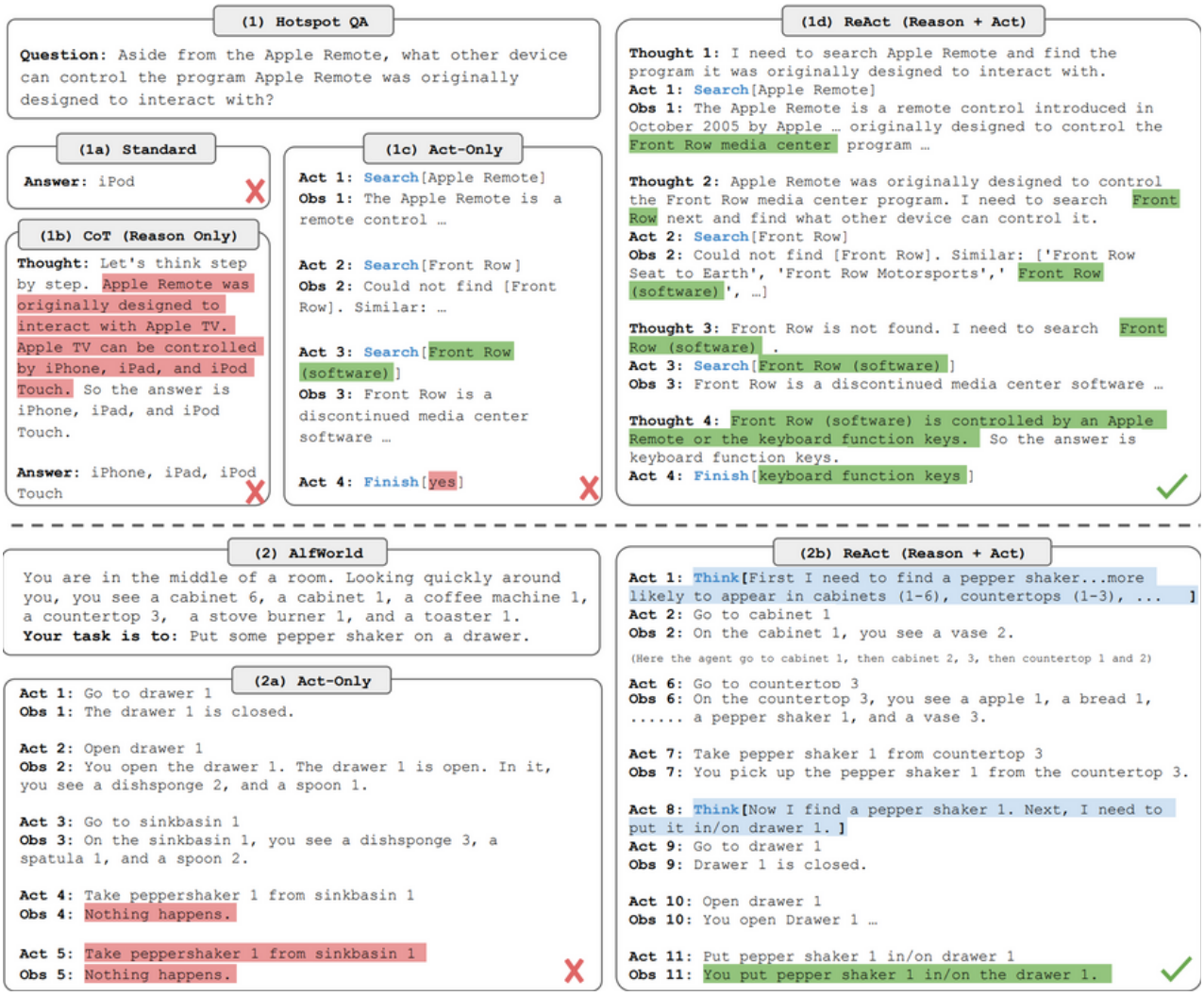
ReAct is said to be better than CoT methods and often suffers from hallucination, resulting in a higher false positive rate and major failure mode. In contrast, ReAct is more grounded, fact-driven, and trustworthy, thanks to access to an external knowledge base.

ReAct addresses these limitations by creating a synergy between reasoning and acting. It prompts LLMs to generate verbal reasoning traces and actions related to a task. This allows the model to dynamically reason, create, maintain, and adjust plans for action. Moreover, it enables interaction with external environments, like Wikipedia, to incorporate additional information into the reasoning process.

The method is executed by a chain of prompts with the following structure:
Thought: This is a reasoning step where the model, or agent, thinks about the task at hand. It's a form of internal dialogue that the model uses to reason about the current context, which includes previous observations and actions. The thought doesn't affect the external environment, but it does update the model's understanding of the task. For example, the model might have a thought like "The question is asking about historical events, so I should look for information in history articles."
Action: This is a step where the model takes an action based on its thought. The action could be something like searching for information, moving to a new location, or answering a question. In the context of ReAct, an action is often an interaction with an external environment, such as querying a Wikipedia API for information. The action taken by the model is designed to help it achieve its task.
Observation: This is the feedback that the model receives from the environment after it takes an action. The observation updates the context for future reasoning and actions. For example, if the model's action was to search for information on Wikipedia, the observation might be the information it finds. This new information then becomes part of the context that the model uses for its next thought.

This cycle of thought-action-observation continues until the task is completed. This process not only improves the model's performance on diverse tasks but also makes it more interpretable and controllable for humans.

For example, in a question-answering task, ReAct would first observe the question, reason about what information is needed to answer it, take an action like searching for that information on Wikipedia, and then observe the result of that search. This process would continue until the model has enough information to answer the question.



read more

📝 XYao, Shunyu, et al. "React: Synergizing reasoning and acting in language models." arXiv preprint arXiv:2210.03629 (2022)..
💻 https://github.com/ysymyth/ReAct

Automatic Prompt Engineering (APE)

Automatic Prompt Engineer (APE) is a method proposed for automatic instruction generation and selection to steer LLMs towards desired behaviours. It treats the instruction as a "program" optimized by searching over a pool of instruction candidates proposed by an LLM to maximize a chosen score function.

The APE algorithm has two key components: proposal and scoring.

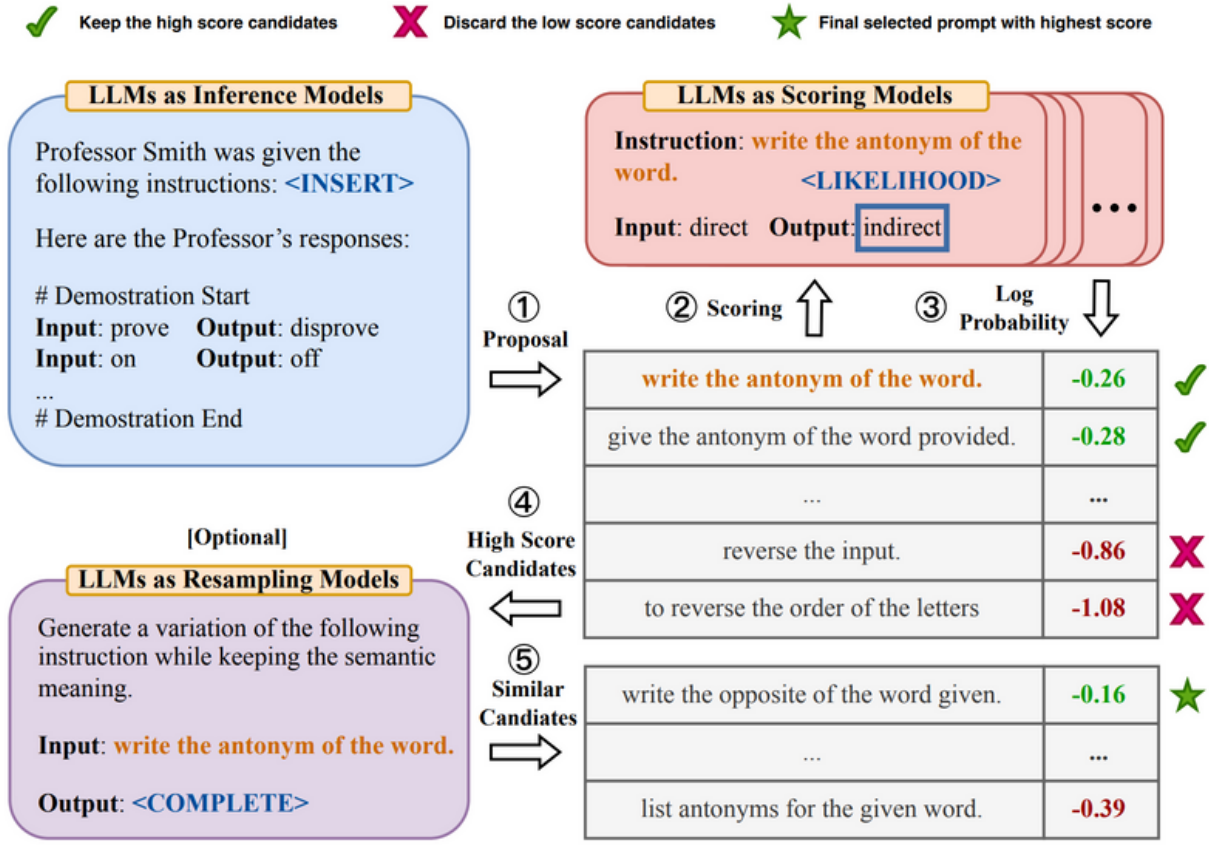
In the **proposal component**, APE uses a pre-trained LLM to propose a few candidate prompts. LLM is given a set of inputs and outputs and its role is to generate a set of instructions that someone could have followed to create output from the inputs. The model is given the same prompt a few times to come up with different instructions.

In the **scoring component**, APE filters and refines the candidate set according to a chosen score function, ultimately choosing the instruction with the highest score. Another LLM is given the instructions from previous steps as well as the inputs. We can then measure how good the instructions are based on how often the LLM generates the desired output while following the instructions.

We can also use an optional **resampling component** that takes the instruction with the best scores and generated variations of them that can potentially result in even better scores.

This system works similarly to evolutionary algorithms where an initial population of prompts from the proposal component is scored against a fitness function by the scoring component and the resampling component introduces mutations.

APE-engineered prompts have been shown to improve few-shot learning performance, find better zero-shot chain-of-thought prompts, as well as steer models toward truthfulness and/or informativeness.



read more

📝 Zhou, Yongchao, et al. "Large language models are human-level prompt engineers." arXiv preprint arXiv:2211.01910 (2022)
💻 https://github.com/keirp/automatic_prompt_engineer

Generated Knowledge Prompting

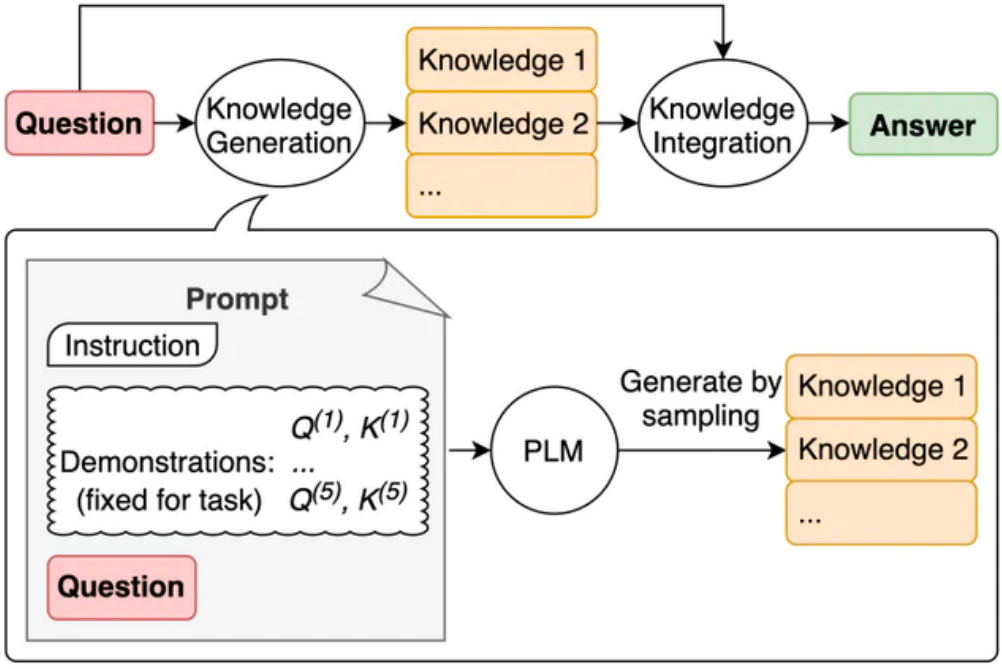
This method generates knowledge from a language model and then uses this knowledge as additional input when answering a question. It doesn't require task-specific supervision for knowledge integration or access to a structured knowledge base

Unlike other methods that add generated text to an inference model, this method uses few-shot demonstrations to prompt knowledge generation.

Few-shot demonstrations: This method uses a few examples (demonstrations) to guide the language model in generating knowledge. These demonstrations consist of a question in the style of the task and a knowledge statement that is helpful for answering this question. For a given task, five demonstrations are written using a specific format.

Generation of knowledge: The demonstrations are used to prompt the language model to generate knowledge statements related to the question. These knowledge statements are then used as additional input when answering the question.

Selection of knowledge: The method involves using a second language model to make predictions with each knowledge statement, then selecting the highest-confidence prediction



example

Suppose we have a commonsense reasoning task where the question is: "Why do plants need sunlight?"

Few-shot demonstrations**:** We would first create a few demonstrations that consist of similar questions and helpful knowledge statements. For example:

- Demonstration 1:
  - Question: "Why do plants need water?"
  - Knowledge Statement: "Plants need water as it is a crucial component of photosynthesis, the process by which they create food."
- Demonstration 2:
  - Question: "Why do plants need carbon dioxide?"
  - Knowledge Statement: "Plants need carbon dioxide as it is used in photosynthesis to produce glucose, which is a type of sugar that plants use for energy."

Generation of knowledge**:** We would then use these demonstrations to prompt the language model (like GPT-3) to generate a knowledge statement related to our original question ("Why do plants need sunlight?"). The language model might generate a statement like: "Plants need sunlight because it provides the energy necessary for photosynthesis, the process by which plants convert water and carbon dioxide into glucose for food."

Answering the question**:** This generated knowledge statement would then be used as additional input when answering the question. A second language model would make predictions based on this knowledge statement and the original question, and then select the highest-confidence prediction as the answer. The answer might be something like: "Plants need sunlight to perform photosynthesis, which is the process they use to convert water and carbon dioxide into glucose for food and oxygen as a byproduct."

read more

📝 Liu, Jiacheng, et al. "Generated knowledge prompting for commonsense reasoning." arXiv preprint arXiv:2110.08387 (2021).
💻 https://github.com/liujch1998/GKP

Tree-of-thought prompting

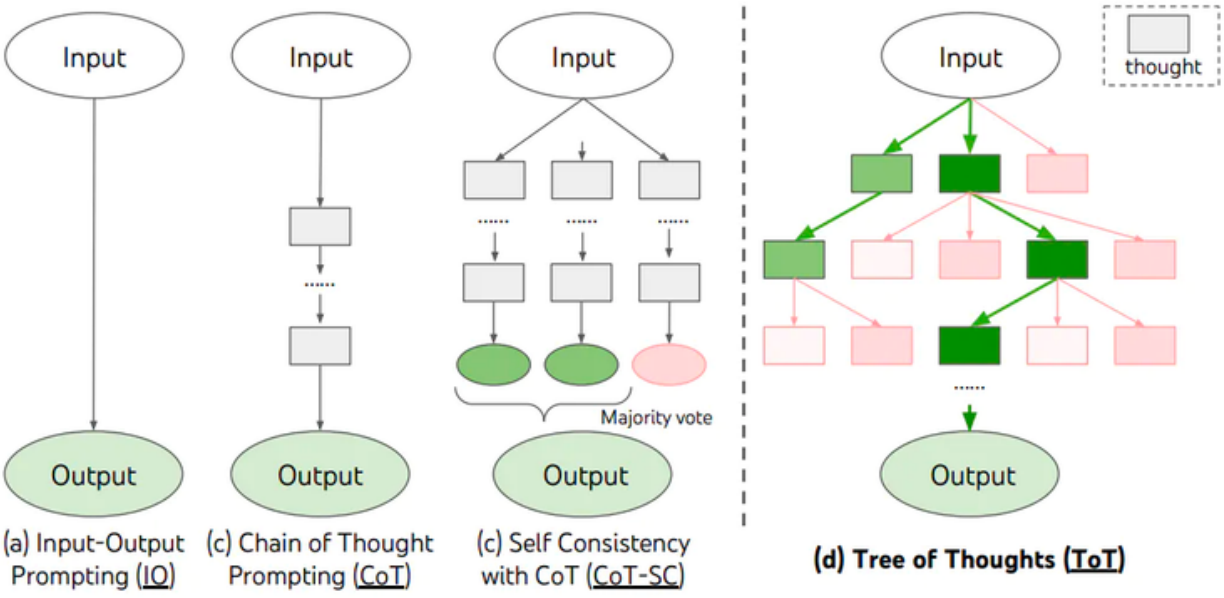
The "Tree of Thoughts" (ToT) method is a problem-solving approach that frames any problem as a search over a tree, where each node represents a partial solution with the input and the sequence of thoughts so far. It's designed to allow language models to explore multiple reasoning paths over thoughts. ToT method consists of 4 steps:

Thought Decomposition: The first step in the ToT method is to break down the intermediate process into thought steps. Depending on the problem, a thought could be a couple of words, a line of equation, or a whole paragraph of a writing plan. The thought should be small enough for the language model to generate promising and diverse samples, yet big enough for the model to evaluate its prospect toward problem solving.
Thought Generation: Given a tree state, the method considers two strategies to generate candidates for the next thought step. One strategy is to sample independent and identically distributed (i.i.d.) thoughts from a CoT prompt. This works better when the thought space is rich, and i.i.d. samples lead to diversity. The other strategy is to propose thoughts sequentially using a "propose prompt". This works better when the thought space is more constrained, so proposing different thoughts in the same context avoids duplication.
State Evaluation: The state evaluator evaluates the progress made towards solving the problem by different states. This serves as a heuristic for the search algorithm to determine which states to keep exploring and in which order.
Search Algorithm: The choice of search algorithm is also an important part of the ToT method. The algorithm is used to navigate the tree of thoughts and guide the problem-solver towards a solution.

let's break down the "Tree of Thoughts" (ToT) method with a simple example. Imagine you're trying to write a story.

Thought Decomposition is like breaking down your story into smaller parts or chapters. Each chapter is a 'thought' in the ToT method. It's a manageable piece of the overall problem (writing a story) that you can work on.

Thought Generation: Now, for each chapter, you need to come up with ideas or 'thoughts'. You could do this in two ways. One way is to brainstorm a bunch of ideas all at once (like throwing darts at a dartboard). This is like the first strategy in the ToT method, which works well when you have a lot of freedom in what you can write. The other way is to come up with ideas one by one, building on the previous idea. This is like the second strategy in the ToT method, which works better when you have a more specific topic for your story.
State Evaluation: This is like reviewing each chapter after you've written it. You're evaluating how well it contributes to the overall story. If a chapter doesn't work well, you might decide to rewrite it or change the order of the chapters.
Search Algorithm: Finally, you need to decide on the order of the chapters and how they link together to form the overall story. This is like navigating the 'tree' of chapters or 'thoughts' to find the best way to tell your story.



read more

📝 Yao, Shunyu, et al. "Tree of thoughts: Deliberate problem solving with large language models." arXiv preprint arXiv:2305.10601 (2023).
📝 Long, Jieyi. "Large Language Model Guided Tree-of-Thought." arXiv preprint arXiv:2305.08291 (2023).

💻 github.com/princeton-nlp/tree-of-thought-llm
💻 How to simply use ToT with chatgpt: github.com/dave1010/tree-of-thought-prompting

Evaluating models

Evaluating solutions based on large language models like GPT-3 can be challenging for several reasons:

Complexity of the Model: Large language models are complex and have millions, or even billions, of parameters. This makes it difficult to understand exactly how they are making decisions or generating responses.

Lack of Explainability: These models are often referred to as "black boxes" because it's hard to understand why they produce the outputs they do. This lack of transparency can make it difficult to evaluate their performance in a meaningful way.

Data Sensitivity: The performance of these models is highly dependent on the data they were trained on. If the training data is biased or unrepresentative in some way, the model's outputs will likely reflect those same biases or gaps in representation. Evaluating these models requires a deep understanding of their training data, which is often not fully accessible.

Context Sensitivity: These models are sensitive to the input they receive. A slight change in phrasing or context can lead to drastically different outputs. This makes it hard to evaluate their performance across a wide range of inputs.

Evaluation Metrics: Traditional evaluation metrics for language models, like perplexity, don't always correlate well with human judgment of quality. Newer metrics like BLEU, ROUGE, and METEOR are better, but still imperfect.

Long-Term Dependability: It's hard to predict how these models will perform on unseen data or in new contexts over time. This makes it challenging to evaluate their long-term reliability and usefulness.

Resource Intensive: Evaluating large language models can be computationally expensive and time-consuming, especially when trying to assess performance across a wide range of tasks or domains.

Evaluating solutions based on LLMs that have been fine-tuned in a zero-shot and few-shot learning setting presents additional challenges beyond those associated with evaluating large language models in general. Here are some reasons why it's difficult:

Lack of Training Data: In zero-shot learning, the model is expected to generalize to tasks for which it has seen no training examples. This makes it difficult to evaluate how well the model will perform on these tasks, as there's no ground truth data to compare against.

Task Ambiguity: Without specific training examples, it can be unclear what the exact task or objective is. This ambiguity makes it hard to define a clear evaluation metric.

Model Bias: Since the model is not trained on task-specific data, it relies heavily on the biases and patterns learned during pre-training. This can lead to unexpected or undesirable outputs, making evaluation challenging.

Inconsistency: The model's performance can be highly variable across different tasks, as it's trying to generalize from unrelated training data. This inconsistency makes it hard to evaluate the model's overall performance.

Prompts Design: In zero-shot learning, the design of the prompts given to the model can significantly impact its performance. Evaluating the model's performance therefore also requires evaluating the quality of the prompts, which can be subjective and difficult to quantify.

Unpredictability: Without task-specific training, the model's outputs can be unpredictable and difficult to interpret. This unpredictability complicates the evaluation process.

Overfitting to Pretraining Data: Since the model is not fine-tuned on task-specific data, it might overfit to the patterns in the pretraining data, which could lead to poor generalization to the new tasks.

Evaluation Metrics: As with large language models in general, finding suitable evaluation metrics that correlate well with human judgment of quality is a challenge. This is even more difficult in a zero-shot learning context, where the tasks can be very diverse.

Self-consistency

SelfCheckGPT is a method designed to detect whether responses generated by Large Language Models (LLMs) like GPT-3 are factual or hallucinated (made up). It's a zero-resource, black-box approach, meaning it doesn't require any external databases or access to the inner workings of the model.
The fundamental idea behind SelfCheckGPT is that when a language model understands a concept well, the responses it generates about that concept will be similar and contain consistent facts. However, if the model is making up or "hallucinating" facts, the responses it generates will likely diverge and contradict each other.
Here's how it works:

- SelfCheckGPT generates multiple responses from a language model about a particular topic.
- It then compares these responses to see if they are consistent or if they contradict each other.
- If the responses are similar and consistent, it's likely that the model is providing factual information. If the responses diverge and contradict each other, it's likely that the model is hallucinating facts.

There are three variants of SelfCheckGPT for measuring informational consistency: BERTScore, question-answering, and n-gram. The simplest method, SelfCheckGPT with unigram (max), works by picking up the token with the lowest occurrence given all the samples. If this token only appears a few times (or once) in the samples, it is likely non-factual.

read more

📝 Manakul, Potsawee, Adian Liusie, and Mark JF Gales. "Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models." arXiv preprint arXiv:2303.08896 (2023).
🖥️ https://github.com/potsawee/selfcheckgpt

Reflection

The "Reflexion: reinforcement via verbal reflection" method is a technique used in decision-making, programming, and reasoning tasks.

Reflexion works by making an initial plan to solve a task, executing the plan, and then reflecting on the outcome. If the task is not successfully completed, the method self-reflects on the errors and suggests lessons to learn from its mistakes over time. For example, if a task is to examine a mug with a desk lamp, but the task fails because the agent looked for the mug first instead of the lamp, Reflexion would reflect on the sequence of actions and realize that it should have looked for the desk lamp first, then the mug. In the next attempt, it would correct its sequence of actions based on this reflection.

Here's a simplified explanation of each step in the method:
Decision-Making: The first step involves making an initial plan to solve a task. For instance, if the task is to examine a mug with a desk lamp, the initial plan might involve finding and taking a mug, and then finding and using a desk lamp. This plan is then executed.

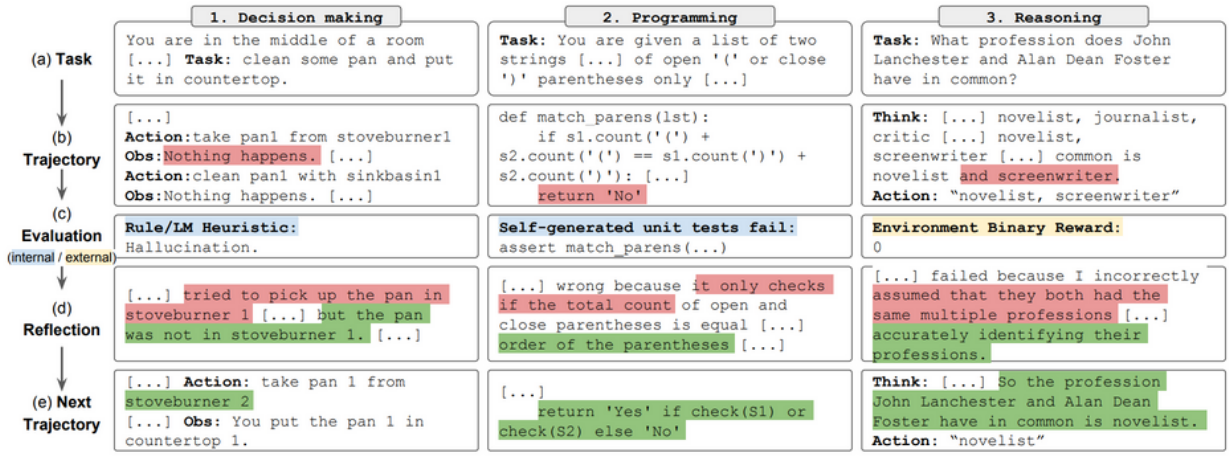
Execution and Feedback: After the initial plan is executed, the system receives feedback on the outcome. If the task is not successfully completed, the system moves to the next step.

Self-Reflection: The system self-reflects on the errors and suggests lessons to learn from its mistakes over time. For example, if the task failed because the system looked for the mug first instead of the lamp, it would reflect on the sequence of actions and realize that it should have looked for the desk lamp first, then the mug.

I**nstruction for Next Attempt**: Based on the self-reflection, the system generates instructions for the next attempt. In the next attempt, it would correct its sequence of actions based on this reflection.

Repeat the Process: The process is repeated until the task is successfully completed. If the task is completed successfully, the status is marked as "Success". If not, the system reflects on the errors, learns from them, and tries again.

In the context of programming, the steps are similar but involve function implementation, unit test feedback, self-reflection, and instruction for the next function implementation.
However, Reflexion struggles with tasks that require a significant amount of diversity and exploration. For example, in an e-commerce website navigation task, Reflexion was unable to generate helpful self-reflections after failed attempts, indicating that it might struggle with tasks that require creative problem-solving.



read more

📝 Shinn, N., Cassano, F., Labash, B., Gopinath, A., Narasimhan, K., and Yao, S., "Reflexion: Language Agents with Verbal Reinforcement Learning", arXiv preprint arXiv.2303.11366 (2023)
🖥️ https://github.com/noahshinn024/reflexion

LangKit

LangKit is an open-source text metrics toolkit for monitoring language models. It offers an array of methods for extracting relevant signals from the input and/or output text, which are compatible with the open-source data logging library whylogs.

The currently supported metrics include:

Text Quality

- readability score
- complexity and grade scores

Text Relevance

- Similarity scores between prompt/responses
- Similarity scores against user-defined themes

Security and Privacy

- patterns - count of strings matching a user-defined regex pattern group
- jailbreaks - similarity scores with respect to known jailbreak attempts
- prompt injection - similarity scores with respect to known prompt injection attacks
- refusals - similarity scores with respect to known LLM refusal of service responses

Sentiment and Toxicity

- sentiment analysis
- toxicity analysis

read more

https://github.com/whylabs/langkit

Tools

LangChain

LangChain is an open-source Python library designed to simplify the process of building applications powered by large language models (LLMs). It provides a variety of features to help developers work with LLMs, including:

Generic Interface to Foundation Models: LangChain provides a common interface to a variety of different foundation models. This allows developers to use different models without having to understand the specifics of each one.

Prompt Management: LangChain provides a framework to help manage prompts, which are the inputs given to LLMs to generate responses. This can be useful for creating more complex interactions with the LLM.

Central Interface to Long-Term Memory and External Data: LangChain provides a central interface to long-term memory, external data, other LLMs, and other agents for tasks an LLM is not able to handle (e.g., calculations or search). This can be useful for building more complex applications that need to interact with various data sources or perform tasks beyond the capabilities of the LLM.

Chaining: LangChain allows developers to combine LLMs with other components to create an application. This can involve combining LLMs with prompt templates, chaining multiple LLMs sequentially, combining LLMs with external data for question answering, and combining LLMs with long-term memory for chat history.

Accessing External Data: LangChain provides a way to give LLMs access to specific external data, which can be useful for applications that need to reference specific documents or emails.

In summary, LangChain is a powerful tool that provides a variety of features to help developers build applications powered by large language models.

read more

https://github.com/hwchase17/langchain

Guidance

Guidance enables you to control modern language models more effectively and efficiently than traditional prompting or chaining. Guidance programs allow you to interleave generation, prompting, and logical control into a single continuous flow matching how the language model actually processes the text.

Guidance is a templating language, that allows variable interpolation (e.g., {{proverb}}) and logical control. But unlike standard templating languages, guidance programs have a well-defined linear execution order that directly corresponds to the token order as processed by the language model. This means that at any point during execution, the language model can be used to generate text (using the {{gen}} command) or make logical control flow decisions. This interleaving of generation and prompting allows for a precise output structure that produces clear and parsable results.

Guidance allows for more control than langchain library, for each field you want LLM to fill you can specify parameters like temperature, allowed values, regex pattern and many more.

example

```
character_maker = guidance("""The following is a character profile for an RPG game in JSON format.
```json
{
    "id": "{{id}}",
    "description": "{{description}}",
    "name": "{{gen 'name'}}",
    "age": {{gen 'age' pattern='[0-9]+' stop=','}},
    "armor": "{{#select 'armor'}}leather{{or}}chainmail{{or}}plate{{/select}}",
    "weapon": "{{select 'weapon' options=valid_weapons}}",
    "class": "{{gen 'class'}}",
    "mantra": "{{gen 'mantra' temperature=0.7}}",
    "strength": {{gen 'strength' pattern='[0-9]+' stop=','}},
    "items": [{{#geneach 'items' num_iterations=5 join=', '}}"{{gen 'this' temperature=0.7}}"{{/geneach}}]
}```""")

# generate a character
character_maker(
    id="e1f491f7-7ab8-4dac-8c20-c92b5e7d883d",
    description="A quick and nimble fighter.",
    valid_weapons=valid_weapons, llm=llama
)
```

read more

https://github.com/microsoft/guidance
```